Table 1: AllowOverride directive values

Directive value	Effect
AuthConfig	Allows directives related to authentication and authorisation.
FileInfo	Allows directives controlling document types.
Indexes	Allows directives controlling directory indexing, i.e., listing of files when a directory is requested.
Limit	Permits the use of Allow, Deny and Order directives.
Options[=Option,...]	Allows directives controlling specific directory features.

Configuration sections

As I told you earlier, the configuration file contains certain sections that resemble HTML mark-up. Here, I list and explain them.

IfModule

This checks if a particular module is loaded into memory (enabled) or not. It basically tells Apache to parse the configuration inside the section only if the module is loaded; else, to skip it. For example:

```
<IfModule fcgid_module>

AddHandler fcgid-script .php

</IfModule>



<IfModule !fcgid_module>

<IfModule fastcgi_module>

AddHandler fastcgi-script .php

</IfModule>

</IfModule>
```

IfModule sections can be nested, as seen above. The bang in *<IfModule !module-identifier>* is used to negate—i.e., include the configuration section if the module is *not* loaded.

IfDefine

Checks if a particular parameter was defined while starting Apache, and is usually used to load modules according to the start-up command, eliminating the need to modify configuration files every now and then. For example:

```
<IfDefine Rewrite>

LoadModule modules/mod_rewrite.so

</IfDefine>
```

In this example, the *rewrite* module will be loaded if Apache was launched with the command:

```
apachectl -D Rewrite start
```

…or,

```
httpd -D Rewrite -k start
```

Directory, Files, FilesMatch, Location, and LocationMatch

These directives are used to control configuration related to the particular elements—Directory, Files, and Location. The difference between them is that *Directory* will match a physical directory but not a symlink (symbolic link). Files will also match symlinks.

Location has no limitations -- it will match a file, location, symlink, alias, etc.

The regular-expression variants for these have *'Match'* appended to the directive. *FilesMatch* is the regular-expression variant of *Files*, and so on. It is possible to use simple shell wild-cards like * and ? in the non regular-expression variants, as follows:

```
<Files ~ .*>

# configuration

</Files>
```

There is no need to use the regular-expression variants to match shell wild-cards, since those variants have more processing overheads, and slow down Apache.

The most important directive in relation to this is the *Options* directive. It controls what features should be enabled, disabled, permitted or not permitted for all the elements matching the regex (files or directories, as applicable). Read *http://j.mp/fzo0Hk* (Apache official documentation) for more information on this. END

By: Nilesh Govindarajan

The author has been using Linux for more than four years. He keeps taking up small programming projects and works on server administration. He can be reached at *contact@nileshgr.com*, or *@nileshgr on Twitter/identi.ca*.